作业提交与管理

在HPC三期(ACD)平台中，作业可以通过Slurm调度器提交至集群，并根据一定规则分配资源调度计算。

> **提示**
>
> 更多关于Slurm命令用法请参考官方文档：Slurm Documentation

作业提交

提交普通作业（命令行模式）

用户可以使用`sbatch <params> <job_name>`命令提交作业，并指定参数

示例

```
$sbatch -p acd_u --input=input.sh -o output_%j.txt -e err_%j.txt -n 8 --gres=gpu:1 job_script.sh
```

> **常用params**：
> **-p acd_u**：指定acd_u队列，全部队列信息请查看：集群队列
> **--input input.sh**：指定作业输入文件
> **-o output_%j.txt**：指定作业标准输出文件，%j为作业号
> **-e err_%j.txt**：指定作业标准错误输出文件
> **-n 8**：指定CPU总核心数
> **--gres 1**：指定GPU卡数
> **-w ACD1-1,ACD1-2**：指定ACD1-1和ACD1-2节点，节点信息请使用`sinfo`查看
> **-x "~ACD1-1"**：排除ACD1-1节点
> **-D /apps**：指定作业执行路径为/apps；默认情况下，没有-D 选项时，作业的执行路径为作业的提交路径

提交普通作业（脚本模式）

除命令行提交外，用户可以通过**脚本提交模式**提交作业，方便用户管理作业提交参数和相关作业参数配置。将提交时需要指定的参数写到脚本中，用户可以批量重复使用该脚本，不用每次都指定参数。

- 脚本提交：`sbatch my_job.sh`
- 脚本格式（my_job.sh）：

my_job.sh

```
#!/bin/bash
#SBATCH -p acd_u          # 指定GPU队列
#SBATCH -o output_%j.txt  # 指定作业标准输出文件，%j为作业号
#SBATCH -e err_%j.txt     # 指定作业标准错误输出文件
#SBATCH -n 8             # 指定CPU总核心数
#SBATCH --gres=gpu:1     # 指定GPU卡数
#SBATCH -D /apps         # 指定作业执行路径为/apps

# 以下是作业要执行的命令
echo "Job started at $(date)"
python your_script.py  # 假设要运行一个Python脚本
echo "Job ended at $(date)"
```

提交并行作业

用户可以提交多节点并行作业

- 脚本提交：`sbatch my_job.sh`
- 脚本格式（my_job.sh）：

示例 my_job.sh

```
#!/bin/bash

#SBATCH -p acd
#SBATCH --job-name=speed_test
#SBATCH -o /data/user/user11/project/PRM2/slurm_log/%j.out # slurm的输出文件，%j是jobid
#SBATCH --nodes=2                # 请求2个节点
#SBATCH --ntasks-per-node=4      #每个节点上4个任务
#SBATCH --gres=gpu:4             # 每个节点4个GPU
```

```
module load anaconda3
module load cuda/12.4

export NCCL_SOCKET_IFNAME=vlan0.2135
export NCCL_NET_GDR_LEVEL=PHB
export NCCL_IB_DISABLE=0
export
NCCL_IB_HCA=mlx5_0,mlx5_1,mlx5_2,mlx5_6,mlx5_7,mlx5_8,mlx5_9,mlx5_10
export NCCL_IB_GID_INDEX=3
export NCCL_IB_TC=138
export NCCL_NSOCKS_PERTHREAD=8
export NCCL_SOCKET_NTHREADS=4
export NCCL_NVLS_ENABLE=0
export NCCL_NVLS_PLUGIN=1
export NCCL_NVLS_LANES=2
export NCCL_IB_QPS_PER_CONNECTION=8
export NCCL_IB_SPLIT_DATA_ON_QPS=1
export NCCL_MIN_CTAS=32
export NCCL_MAX_CTAS=128
export NCCL_IB_RETRY_CNT=7
export NCCL_MIN_NCHANNELS=64
export NCCL_MAX_NCHANNELS=256
export NCCL_NCHANNELS_PER_NET_PEER=32
export NCCL_BUFFSIZE=33554432
export NCCL_LL_BUFFSIZE=33554432
export NCCL_P2P_NET_CHUNKSIZE=2097152
export NCCL_P2P_LEVEL=nvl
export NCCL_ALGO=nvlstree,ring
export NCCL_LL_MAX_NCHANNELS=4
export NCCL_CROSS_NIC=1
export NCCL_IGNORE_CPU_AFFINITY=1
export NCCL_SHM_DISABLE=0
export NCCL_COLLNET_ENABLE=1
export NCCL_DEBUG=INFO
export NCCL_TIMEOUT=3600
export NCCL_IB_TIMEOUT=3600


# 以下是作业要执行的命令
echo "Job started at $(date)"
python your_script.py  # 假设要运行一个Python脚本
echo "Job ended at $(date)"
```

提交数组作业

用户可提交数组作业，以共享相同的可执行文件和资源需求，但是具有不同的输入文件和输出文件。

- 脚本提交：`sbatch my_job.sh`
- 脚本格式（my_job.sh）：

示例 my_job.sh

```
#!/bin/bash
#SBATCH -p normal       # 指定队列
#SBATCH -o output_%A_%a.txt  # 指定标准输出文件，%A 是数组作业的主作业 ID，%a 是当前子作业的索引
#SBATCH -e error_%A_%a.txt   # 指定标准错误输出文件
#SBATCH -n 1            # 指定每个子作业所需的 CPU 核心数
#SBATCH --array=1-10    # 指定数组作业的范围，这里表示从 1 到 10 的 10 个子作业

# 根据子作业索引设置不同的参数
PARAM=$SLURM_ARRAY_TASK_ID

# 执行作业命令，这里以打印参数为例
echo "Running task $PARAM"
python your_script.py $PARAM  # 假设要运行一个 Python 脚本并传入参数
```

参数说明：

- **#SBATCH --array=1-10：** 这行指令定义了数组作业的范围，1-10 表示将创建 10 个子作业，子作业的索引从 1 到 10。你也可以使用逗号分隔不同的索引值，如 --array=1,3,5 表示只创建索引为 1、3、5 的子作业；还可以使用步长，如 --array=1-10:2 表示索引为 1、3、5、7、9 的子作业。
- **$SLURM_ARRAY_TASK_ID：** 这是一个环境变量，在每个子作业中，它的值等于该子作业的索引。你可以根据这个索引来设置不同的参数或输入文件。
- **%A 和 %a：** 在输出文件的命名中，%A 代表数组作业的主作业 ID，%a 代表当前子作业的索引。这样可以确保每个子作业的输出文件不会相互覆盖。

提交交互式作业

交互式作业是一种类前端作业，虽然作业在后端（某一计算节点）执行，但执行的过程和输出会实时呈现到用户提交端，在这一过程中用户也可以参与其中，进行必要的人机交互。

用户可使用`srun`提交交互式作业

示例

```
$ srun -p acd_u -n 4 --mem=8G --gres=gpu:1 --time=01:00:00 --pty bash
```

参数说明：

- **--time=01:00:00：** 设置作业的最长运行时间为 1 小时。当达到这个时间限制时，作业会被自动终止。
- **-pty bash：** 分配一个伪终端并启动 bash shell

> **注意事项**
> 所有作业默认最长运行7天，到期前可在科大Go提交IT工单，申请延长7天

作业查看与管理（命令行）

查看用户作业（等待、运行、挂起）

```
$ squeue -u <username> [-t PENDING,RUNNING,SUSPENDED]
```

查看用户历史作业

```
$ sacct -u <username> [--array]
```

查看作业详情

```
$ scontrol show job <jobid>
```

查看数组作业

```
$ scontrol show job <jobid_$task_id>
```

查看作业PENDING原因

```
$ scontrol show job <jobid>
```

挂起作业

```
$ scontrol suspend <jobid>
```

恢复作业

```
$ scontrol resume <jobid>
```

终止作业

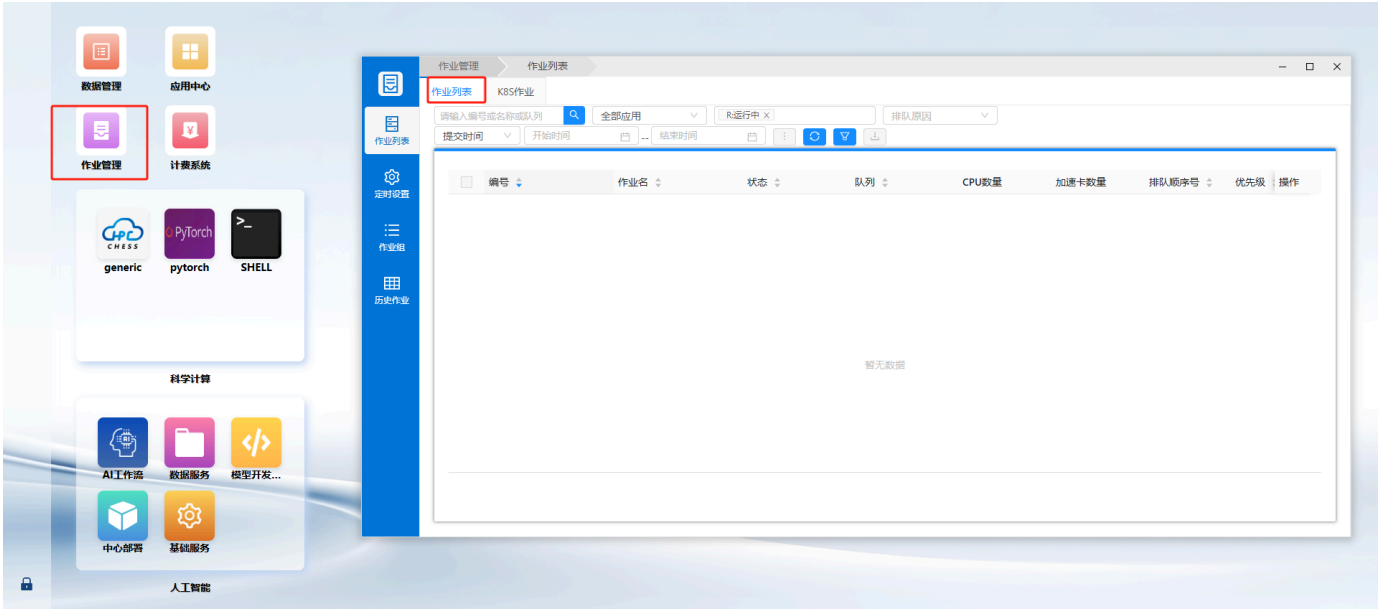
```
$ scancel <jobid>
```

作业查看与管理（WEB）

查看用户作业

登录HPC三期平台门户 --- 作业管理 --- 作业列表

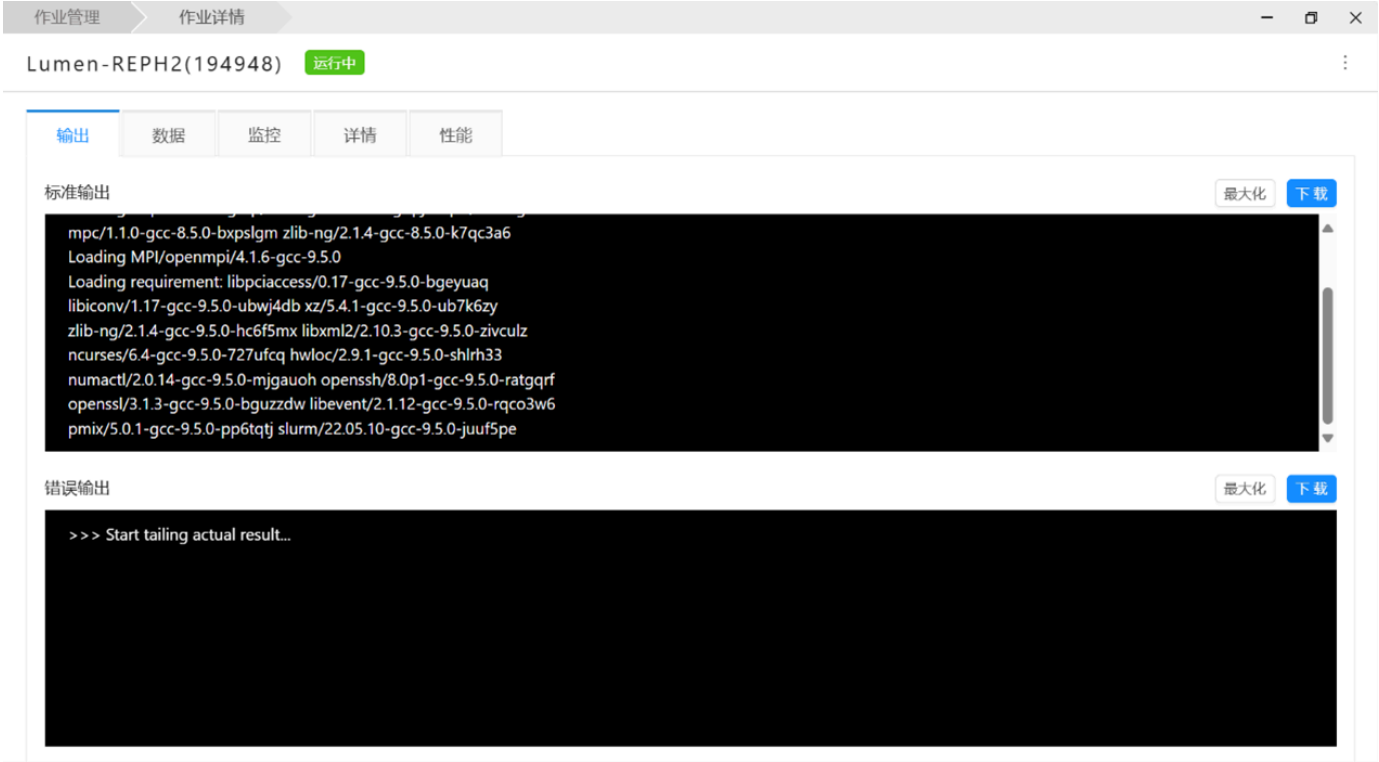
历史作业显示已结束七天以上的作业情况



查看作业详情

点击作业名称，查看对应作业详情，切换：输出，数据，监控，详情，性能查看对应信息

运行中的作业点击“性能”支持查看作业在该节点的内存、CPU、加速卡等资源使用状况



终止作业

选择对应作业，在最右侧【操作】列进行终止操作